Seminar 03 — TypeScript

PB138 — Basics of Web Development

Agenda

1. **TypeScript Type System** — structural typing, unions, narrowing, utility types (~30 min)
2. **tsconfig.json** — quick reference (~5 min)
3. **TypeScript in React** — props, events, generic components (~20 min)
4. **Runtime Validation** — Zod v4 (~20 min)
5. **ts-pattern** — pattern matching (~15 min)
6. **Buffer / Q&A** (~10 min)

> 🛠️ Total coding time: ~25 min across 3 tasks + bonus

Part 1: TypeScript Type System

Structural Typing

TypeScript uses **structural typing** — if it has the right shape, it fits.

```
type Dog = { name: string; bark(): void };
type Pet = { name: string };

const dog: Dog = { name: "Rex", bark() {} };
const pet: Pet = dog; // ✅ Dog has all Pet fields
```

No `implements` needed. Shape is all that matters.

Basic Types

```typescript
// Primitives
const name: string = "Alice";
const age: number = 25;
const active: boolean = true;

// Arrays
const ids: number[] = [1, 2, 3];

// Object types
type User = {
  id: number;
  name: string;
  email?: string; // optional
};
```

Union & Intersection Types

```
// Union — one of
type Status = "loading" | "success" | "error";
type ID = string | number;

// Intersection — combine
type User = { name: string; email: string };
type Admin = User & { permissions: string[] };
```

Literal unions are great for constraining values:

```
function setTheme(theme: "light" | "dark") { ... }
setTheme("light"); // ✅
setTheme("blue");  // ❌ compile error
```

Discriminated Unions

A **tagged union** where each variant has a literal discriminant field.

```
type Result =
  | { status: "success"; data: string }
  | { status: "error"; message: string };

function handle(result: Result) {
  switch (result.status) {
    case "success":
      return result.data;    // TS knows `data` exists
    case "error":
      return result.message; // TS knows `message` exists
  }
}
```

Type Narrowing

TypeScript **narrows** types based on control flow.

```
// typeof
function format(value: string | number) {
  if (typeof value === "string") return value.toUpperCase();
  return value.toFixed(2);
}

// in operator
type Fish = { swim(): void };
type Bird = { fly(): void };

function move(animal: Fish | Bird) {
  if ("swim" in animal) animal.swim();
  else animal.fly();
}
```

Custom Type Guards

```ts
type Cat = { kind: "cat"; meow(): void };
type Dog = { kind: "dog"; bark(): void };
type Animal = Cat | Dog;

function isCat(animal: Animal): animal is Cat {
  return animal.kind === "cat";
}

function handle(animal: Animal) {
  if (isCat(animal)) {
    animal.meow(); // TS knows it's Cat
  }
}
```

Utility Types

Utility	What it does
`Partial<T>`	All fields optional
`Required<T>`	All fields required
`Pick<T, K>`	Only specified fields
`Omit<T, K>`	Everything except specified
`Record<K, V>`	Object with keys K and values V
`Readonly<T>`	All fields readonly

```
type User = { id: number; name: string; email: string };

type UserPreview = Pick<User, "id" | "name">;
type UserUpdate = Partial<Omit<User, "id">>;
```

Generics

```
function first<T>(arr: T[]): T | undefined {
  return arr[0];
}

first([1, 2, 3]);    // number
first(["a", "b"]);   // string
```

```
// With constraints
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

getProperty({ name: "Alice", age: 25 }, "name"); // string
```

✏️ Task 1 — Type System Exercises (~10 min)

1. **Discriminated union:** Define an `ApiResponse<T>` type with three variants:
   - `{ status: "loading" }`
   - `{ status: "success"; data: T }`
   - `{ status: "error"; error: string }`
2. **Utility types:** Given `type Product = { id: number; name: string; price: number; description: string }`, create:
   - `ProductPreview` — only `id` and `name`
   - `ProductUpdate` — all fields optional except `id`
3. **Type guard:** Write a function `isString(value: unknown): value is string` and use it to filter `(string | number)[]` into `string[]`.

Part 2: tsconfig.json

Key Compiler Options

```
{
  "compilerOptions": {
    "strict": true,                    // enable all strict checks
    "target": "ES2022",                // output JS version
    "module": "ESNext",                // module system
    "moduleResolution": "bundler",     // how imports are resolved
    "jsx": "react-jsx",               // JSX transform (no React import needed)
    "baseUrl": ".",
    "paths": { "@/*": ["./src/*"] },   // import aliases
    "noUncheckedIndexedAccess": true,  // array[i] is T | undefined
    "skipLibCheck": true
  }
}
```

`strict: true` enables `strictNullChecks`, `noImplicitAny`, `strictFunctionTypes` and more.

Part 3: TypeScript in React

Typing Props

```tsx
type ButtonProps = {
  label: string;
  variant?: "primary" | "secondary";
  disabled?: boolean;
  onClick: () => void;
};

function Button({ label, variant = "primary", onClick }: ButtonProps) {
  return <button onClick={onClick}>{label}</button>;
}
```

Typing Events

```
function SearchForm() {
  const [query, setQuery] = useState("");

  function handleChange(e: React.ChangeEvent<HTMLInputElement>) {
    setQuery(e.target.value);
  }

  function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
    e.preventDefault();
    console.log("Searching:", query);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={query} onChange={handleChange} />
    </form>
  );
}
```

Generic Components

```tsx
type ListProps<T> = {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
};

function List<T>({ items, renderItem }: ListProps<T>) {
  return <ul>{items.map((item, i) => <li key={i}>{renderItem(item)}</li>)}</ul>;
}

// Usage — T is inferred as User
<List
  items={users}
  renderItem={(user) => <span>{user.name}</span>}
/>
```

✏️ Task 2 — Type a React Component (~5 min)

Given this untyped component, add proper TypeScript types:

```
function StatusCard({ title, status, count, onAction }) {
  return (
    <div>
      <h3>{title}</h3>
      <span>{status === "active" ? "🟢" : "🔴"} {status}</span>
      {count !== undefined && <p>{count} items</p>}
      <button onClick={onAction}>Take Action</button>
    </div>
  );
}
```

Define `StatusCardProps` with: `title` (string), `status` (`"active" | "inactive"`), optional `count` (number), `onAction` (click handler).

Part 4: Runtime Validation — Zod

The Problem

TypeScript types are **erased at runtime**. API responses are untyped.

```
const res = await fetch("/api/users");
const data = await res.json(); // any 😱
```

You need **runtime validation** to ensure data matches your types.

Zod v4 Basics

```
import { z } from "zod/v4";

const UserSchema = z.object({
  id: z.number(),
  name: z.string().min(1),
  email: z.string().email(),
  role: z.enum(["admin", "user"]),
  age: z.number().min(0).optional(),
});

// Auto-generate the TypeScript type
type User = z.infer<typeof UserSchema>;
```

safeParse & Error Handling

```
const result = UserSchema.safeParse(data);

if (result.success) {
  console.log(result.data); // fully typed User
} else {
  console.log(result.error.issues);
  // [{ path: ["email"], message: "Invalid email" }]
}
```

`safeParse` never throws — returns `{ success, data }` or `{ success, error }`.

For quick validation, `parse()` throws on invalid input:

```
const user = UserSchema.parse(data); // throws ZodError if invalid
```

Composing Schemas

```
const AddressSchema = z.object({
  street: z.string(),
  city: z.string(),
  zip: z.string().regex(/^\d{5}$/),
});

const UserWithAddress = UserSchema.extend({
  address: AddressSchema,
});

// Partial, pick, omit — like utility types
const UserUpdate = UserSchema.partial().required({ id: true });
const LoginInput = UserSchema.pick({ email: true }).extend({
  password: z.string().min(8),
});
```

✏️ Task 3 — Zod Validation (~10 min)

Build a registration form validator:

1. Define a `RegistrationSchema` with:
   - `username` : string, min 3 chars, max 20
   - `email` : valid email
   - `password` : string, min 8 chars
   - `age` : number, min 13, optional
2. Validate this input and log errors:

```
const input = { username: "ab", email: "not-an-email", password: "123" };
```

3. Create a `LoginSchema` by picking `email` from your schema and adding a `password` field.
4. Infer the TypeScript types using `z.infer<>` .

Part 5: ts-pattern

Pattern Matching

`ts-pattern` brings **exhaustive pattern matching** to TypeScript.

```
import { match } from "ts-pattern";

type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "rect"; w: number; h: number };

const area = (shape: Shape) =>
  match(shape)
    .with({ kind: "circle" }, (s) => Math.PI * s.radius ** 2)
    .with({ kind: "rect" }, (s) => s.w * s.h)
    .exhaustive(); // compile error if a case is missing
```

Real-world: Matching API States

```
type ApiState<T> =
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; error: string };

function renderState(state: ApiState<string[]>) {
  return match(state)
    .with({ status: "loading" }, () => <Spinner />)
    .with({ status: "error" }, ({ error }) => <p>Error: {error}</p>)
    .with({ status: "success" }, ({ data }) => (
      <ul>{data.map((item) => <li key={item}>{item}</li>)}</ul>
    ))
    .exhaustive();
}
```

Advanced Patterns

```
import { match, P } from "ts-pattern";

// Wildcards and guards
match(value)
  .with({ type: "user", age: P.number.gte(18) }, (v) => "adult user")
  .with({ type: "user" }, (v) => "minor user")
  .with({ type: P.string }, (v) => `other: ${v.type}`)
  .otherwise(() => "unknown");
```

`P.string`, `P.number`, `P.array(...)` — typed wildcard patterns.

`.exhaustive()` vs `.otherwise()` — compile-time safety vs fallback.

🚀 Bonus — For Fast Finishers

1. **ts-pattern + Zod:** Parse an API response with Zod, then use `match` on the result:

```
const result = UserSchema.safeParse(data);
const message = match(result)
  .with({ success: true }, (r) => `Welcome, ${r.data.name}!`)
  .with({ success: false }, (r) => `Errors: ${r.error.issues.length}`)
  .exhaustive();
```

2. **Refactor:** Take any `if/else` or `switch` chain from your previous tasks and rewrite it using `match()`.

Summary

Concept	Key Idea
Structural typing	Shape compatibility, no `implements` needed
Discriminated unions	Tagged variants with literal discriminant
Type narrowing	TS narrows types via control flow
Utility types	`Partial`, `Pick`, `Omit`, `Record`
React + TS	Type props, events (`ChangeEvent`, `FormEvent`), generics
Zod v4	Runtime validation, `safeParse`, `z.infer<>`
ts-pattern	Exhaustive pattern matching with `match().with().exhaustive()`

Useful Links

- TypeScript Handbook
- TypeScript Playground
- Zod v4 Docs
- ts-pattern Docs
- React TypeScript Cheatsheet

Next Seminar Preview

Seminar 04 — TBD